

Sistema de Produção de Chão de Fábrica - Documentação Técnica

Índice

1. [Visão Geral do Sistema](#)
2. [Arquitetura](#)
3. [Documentação de Componentes](#)
4. [Camada de Serviços](#)
5. [Modelos de Dados](#)
6. [Referência da API](#)
7. [Gestão de Estado](#)
8. [Pontos de Integração](#)
9. [Segurança](#)
10. [Considerações de Performance](#)

Visão Geral do Sistema

Objetivo

O Sistema de Produção de Chão de Fábrica é um sistema de execução de manufatura (MES) em tempo real, projetado para rastrear ordens de produção, consumo de materiais e movimentos de inventário num ambiente de produção.

Funcionalidades Principais

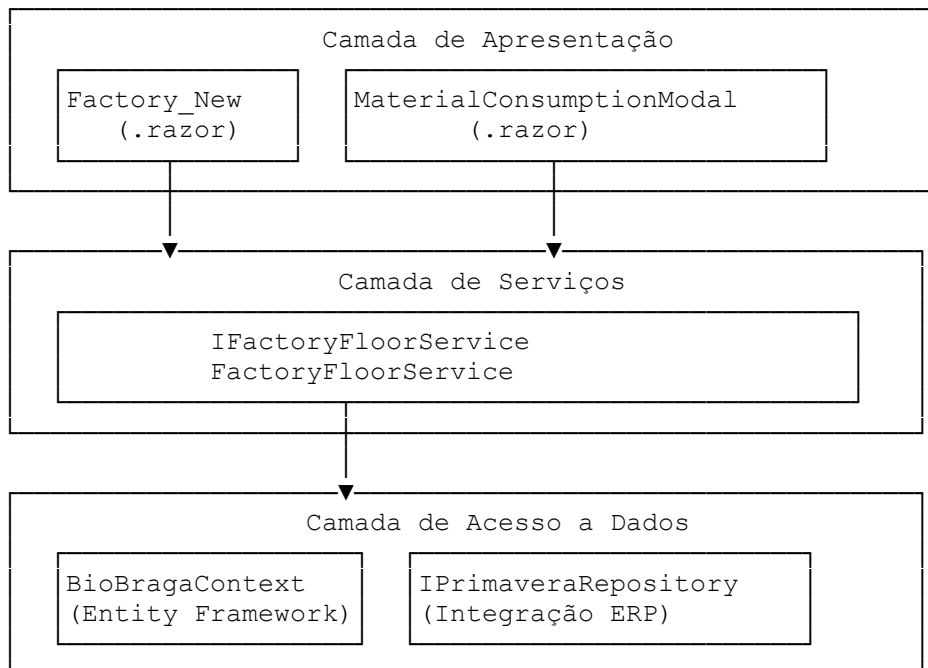
- Gestão de ordens de produção em tempo real
- Rastreamento de consumo de materiais com análise de variância
- Geração automática de movimentos de stock
- Análise de padrões históricos de consumo
- Suporte para produção multi-máquina
- Integração com sistema ERP Primavera

Stack Tecnológica

- **Frontend:** Blazor Server (.NET 8)
- **Framework UI:** MudBlazor
- **Backend:** ASP.NET Core
- **Base de Dados:** Entity Framework Core
- **Integração Externa:** Primavera ERP

Arquitetura

Diagrama de Arquitetura do Sistema



Padrões de Design

- **Padrão Repository:** Abstração de acesso a dados
- **Padrão Service Layer:** Encapsulamento de lógica de negócio
- **Injeção de Dependência:** Container IoC para baixo acoplamento
- **Unit of Work:** Gestão de transações via EF Core

Documentação de Componentes

Factory_New.razor

Objetivo

Interface principal do chão de fábrica fornecendo visualização em tempo real de ordens de produção e mecanismos de controlo.

Estrutura do Componente

```
@page "/factory_new"
@page "/factory_new/{maquinaid}"
@layout FactoryLayout
```

Propriedades Principais

Propriedade	Tipo	Descrição
maquinaid	string?	Parâmetro de rota para filtragem de máquina
CurrentWorker	CrমUtilizador?	Parâmetro em cascata para trabalhador autenticado
allProductionOrders	List<OrdemProducao>	Coleção completa de ordens
filteredProductionOrders	List<OrdemProducao>	Ordens filtradas por máquina
selectedMachine	CrমEquipamento?	Máquina atualmente selecionada
todayStats	ProductionStatsDto?	Métricas de produção de hoje

Métodos do Ciclo de Vida

OnInitializedAsync()

```
protected override async Task OnInitializedAsync()
{
    await LoadInitialData();
    isLoading = true;
}
```

- Carrega máquinas e ordens de produção
- Define flag de dados iniciais carregados

OnParametersSetAsync()

```
protected override async Task OnParametersSetAsync()
```

- Gere mudanças de parâmetros URL
- Gere navegação do browser (voltar/avançar)
- Atualiza dados filtrados baseado na seleção de máquina

Métodos Principais

SelectMachine(string? machineId)

- Atualiza URL para bookmarking
- Filtra ordens localmente sem roundtrip ao servidor
- Atualiza estatísticas de produção

RequestPinForAction(OrdemProducao order, ProductionAction action)

- Abre diálogo de autenticação PIN
- Valida credenciais do trabalhador
- Executa ação de produção após autenticação bem-sucedida

ExecuteProductionAction(OrdemProducao order, ProductionAction action)

- Processa ações Iniciar/Pausar/Parar

- Abre modal de consumo de material para Pausar/Parar
- Atualiza estado da ordem via camada de serviço

MaterialConsumptionModal.razor

Objetivo

Diálogo modal para registrar consumo real de materiais e quantidades de produção.

Funcionalidades Principais

- Carregamento dinâmico de materiais baseado na quantidade de produção
- Cálculo de variância em tempo real
- Exibição de padrões históricos de consumo
- Validação de disponibilidade de stock
- Indicador de progresso de submissão multi-passo

Propriedades do Componente

Propriedade	Tipo	Descrição
OrdemProducao	OrdemProducao?	Ordem de produção a processar
WorkerId	int	ID do trabalhador autenticado
QuantityProduced	decimal	Quantidade real produzida
Waste	decimal	Quantidade de desperdício
Materials	List<MaterialConsumptionDto>	Lista de consumo de materiais

Estratégia de Carregamento de Materiais

```
private async Task OnQuantityProducedChanged(decimal value)
{
    // Mecanismo de debounce com 1.5s de atraso
    _loadMaterialsCts?.Cancel();
    _loadMaterialsCts = new CancellationTokenSource();
    await Task.Delay(1500, token);

    if (!token.IsCancellationRequested)
        await LoadMaterials();
}
```

Processo de Submissão

1. Validar dados
2. Atualizar quantidades da ordem
3. Gerar movimentos de stock
4. Registrar consumo de material
5. Enviar para Primavera ERP
6. Finalizar transação

Rastreamento de Progresso

```
private async Task UpdateProgress(int step, string message)
{
    CurrentStepNumber = step;
    CurrentStep = message;
    StateHasChanged();
    await Task.Yield(); // Permitir atualização UI
}
```

Camada de Serviços

Interface IFactoryFloorService

Operações Principais

Gestão de Ordens de Produção

```
Task<List<OrdemProducao>> GetProductionOrdersAsync(string? maquinaId =
null);
Task<OrdemProducao?> GetProductionOrderByAsync(int orderId,
BioBragaContext? existingContext = null);
```

Controlo de Produção

```
Task<(bool success, string message)> StartProductionAsync(int orderId, int
userId);
Task<(bool success, string message)> PauseProductionAsync(int orderId, int
userId);
Task<(bool success, string message)> EndProductionAsync(int orderId, int
userId);
```

Gestão de Materiais

```
Task<List<MaterialConsumptionDto>> GetMaterialsForProductionAsync(int
orderId, decimal quantityProduced);
Task<(bool success, string message)> RegisterProductionAsync(
    int orderId, int userId, decimal quantityProduced,
    decimal waste, List<MaterialConsumptionDto> actualConsumption);
```

Implementação FactoryFloorService

Injeção de Dependências

```
public FactoryFloorService(
    IDbContextFactory<BioBragaContext> contextFactory,
    IPrimaveraRepository primaveraRepo,
    ILogger<FactoryFloorService> logger,
    IConfiguration configuration)
```

Algoritmo de Cálculo de Material

Lógica GetMaterialsForProductionAsync

```
// Cálculo principal para consumo proporcional de material
```

```

var totalPlannedForOrder = mat.gf.Quantidade ?? 0;
var orderQuantity = order.Quantidade ?? 1;
var perUnitQuantity = totalPlannedForOrder / orderQuantity;
var proportionalQty = Math.Round(perUnitQuantity * quantityProduced, 6);

// Consideração de média histórica
decimal? suggestedQty = null;
if (mat.gf.QuantidadeMedia.HasValue && mat.gf.QuantidadeMedia.Value > 0)
{
    suggestedQty = Math.Round(mat.gf.QuantidadeMedia.Value *
quantityProduced, 6);
}

```

Gestão de Transações

Fluxo de Transação RegisterProductionAsync

```

using var transaction = await context.Database.BeginTransactionAsync();
try
{
    // 1. Atualizar quantidades da ordem
    order.QuantidadeFeita += quantityProduced;
    order.QuantidadeFalta -= quantityProduced;

    // 2. Atualizar rastreamento de consumo
    gastoFase.QuantidadeConsumida += material.ActualQuantity;
    gastoFase.NumeroProducoes++;

    // 3. Gerar movimentos de stock
    await GenerateStockMovementsAsync(...);

    // 4. Confirmar transação
    await transaction.CommitAsync();
}
catch
{
    await transaction.RollbackAsync();
    throw;
}

```

Geração de Movimentos de Stock

Padrão de Geração de Documentos

1. **Entrada de Stock (ENTSTOCK):** Regista produtos acabados
2. **Saída de Stock (SAIDASTOCK):** Regista consumo de material
3. **Documento de Desperdício (DESP):** Regista desperdício de produção

```

private async Task GenerateStockEntryAsync(
    BioBragaContext context,
    OrdemProducao order,
    int userId,
    decimal quantity)
{
    // Criação do cabeçalho do documento
    var doc = new CrmDocumento
    {
        Tipodocumentoid = tipoDoc.Id,

```

```

        Numero = await GetNextDocumentNumber(),
        Data = DateTime.Now,
        DocumentoOrdemFabricao = order.op.Id
    };

    // Linhas do documento para produto e referência
}

```

Modelos de Dados

DTOs Principais

MaterialConsumptionDto

```

public class MaterialConsumptionDto
{
    public int GastosFaseId { get; set; }
    public int? ArtigoId { get; set; }
    public string ArtigoNome { get; set; }
    public string ArtigoCodigo { get; set; }
    public string Unidade { get; set; }

    // Quantidades
    public decimal PlannedQuantity { get; set; }
    public decimal ActualQuantity { get; set; }
    public decimal Stock { get; set; }

    // Análise de Variância
    public decimal Variance => ActualQuantity - PlannedQuantity;
    public decimal VariancePercentage =>
        PlannedQuantity != 0 ? (Variance / PlannedQuantity) * 100 : 0;

    // Dados Históricos
    public decimal? HistoricalAverage { get; set; }
    public decimal? TotalConsumed { get; set; }
    public int? ProductionCount { get; set; }
}

```

ProductionStatsDto

```

public class ProductionStatsDto
{
    public decimal? TotalProduced { get; set; }
    public decimal? TotalPending { get; set; }
}

```

Modelos de Entidade

OrdemProducao (Modelo Composto)

```
public class OrdemProducao
{
    public CrmOrdemProducao op { get; set; }
    public CrmDocumentoDetalhe docline { get; set; }
    public CrmDocumento doc { get; set; }
    public CrmArtigo artigo { get; set; }
    public List<ComponenteProd> ListaComponentes { get; set; }
}
```

Referência da API

Registro de Serviços

```
// Program.cs ou Startup.cs
services.AddScoped<IFactoryFloorService, FactoryFloorService>();
services.AddDbContextFactory<BioBragaContext>(options =>
    options.UseSqlServer(connectionString));
```

Padrões de Endpoints

- /factory_new - Vista geral de produção
- /factory_new/{maquinaid} - Vista específica de máquina

Gestão de Estado

Fluxo de Estado do Componente

Ação Utilizador → Evento Componente → Chamada Serviço → Atualização BD → Refresh UI

↑

↓

StateHasChanged() ←

Parâmetros em Cascata

```
[CascadingParameter] public CrmUtilizador? CurrentWorker { get; set; }
[CascadingParameter] public EventCallback<CrmUtilizador?> OnWorkerChanged {
    get; set; }
```

Persistência de Estado

- Estado baseado em URL para seleção de máquina
- Autenticação de trabalhador baseada em sessão
- Estado de produção persistido em base de dados

Pontos de Integração

Integração Primavera ERP

```
await _primaveraRepo.SendMovimentacaoStock(doc, lines);
```

Gestão de Contexto de Base de Dados

```
// Padrão de reutilização de contexto para transações
var shouldDisposeContext = existingContext == null;
var context = existingContext ?? await
_contextFactory.CreateDbContextAsync();
try
{
    // Operações
}
finally
{
    if (shouldDisposeContext)
        await context.DisposeAsync();
}
```

Segurança

Fluxo de Autenticação

1. Autenticação de trabalhador baseada em PIN
2. Armazenamento de PIN com hash (`Utils.Hash(pin)`)
3. Validação de utilizador ativo
4. Requisitos de autenticação ao nível da ação

Verificações de Autorização

```
if (CurrentWorker == null) return;
if (worker != null && worker.Isactive != true) return null;
```

Trilha de Auditoria

- Todas as ações registadas com ID de utilizador e timestamp
- Logs de produção armazenados em `CrmOrdemProducaoLog`
- Criação de documentos rastreada com campo `CriadoPor`

Considerações de Performance

Estratégias de Otimização

Debouncing

```
// Debounce de 1.5 segundos para carregamento de materiais
await Task.Delay(1500, token);
```

Eager Loading

```
var query = from op in context.CrmOrdemProducao
             join docDetail in context.CrmDocumentoDetalhes...
             // Múltiplos joins para dados completos
```

Filtragem Local

```
private void FilterOrdersByMachine()
{
    // Filtragem client-side para evitar roundtrip ao servidor
    filteredProductionOrders = allProductionOrders
        .Where(o => o.op.Equipamentoid.ToString() == maquinaid)
        .ToList();
}
```

Considerações de Escalabilidade

- Pool de conexões de base de dados via IDbContextFactory
- Operações assíncronas em todo o sistema
- Minimização de âmbito de transações
- Queries de base de dados indexadas em campos frequentemente acedidos

Pontos de Monitorização

- Performance de query de ordens de produção
- Tempo de resposta de cálculo de materiais
- Duração de geração de movimentos de stock
- Latência de integração Primavera

Tratamento de Erros

Padrão da Camada de Serviço

```
try
{
    // Operação
    return (true, "Mensagem de sucesso");
}
catch (Exception ex)
{
    _logger.LogError(ex, "Descrição do erro");
    return (false, $"Mensagem de erro: {ex.Message}");
}
```

Feedback de Erro na UI

```
Snackbar.Add($"Mensagem de erro", Severity.Error);
```

Considerações de Deployment

Requisitos de Configuração

- Connection strings para base de dados BioBraga
- Endpoints da API Primavera
- Configuração de logging
- Configuração de caminho de armazenamento de imagens

Variáveis de Ambiente

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=...;Database=BioBraga;..."
  },
  "Primavera": {
    "ApiEndpoint": "https://..."
  }
}
```

Health Checks

- Conectividade de base de dados
- Disponibilidade da API Primavera
- Responsividade da camada de serviço

Manutenção e Suporte

Estratégia de Logging

- Logging estruturado com IDs de correlação
- Métricas de performance para operações chave
- Rastreamento de erros com stack traces
- Trilha de auditoria de ações do utilizador

Manutenção de Base de Dados

- Otimização regular de índices
- Arquivo de logs históricos de produção
- Procedimentos de reconciliação de níveis de stock
- Limpeza de ordens canceladas/concluídas

Controlo de Versões

- Versionamento semântico para releases
- Scripts de migração de base de dados
- Considerações de compatibilidade retroativa
- Implementação de feature flags para rollouts graduais